

VxLEARN Networks

Networking & Cybersecurity Track
Simulated Employment Program

Lab Report: Linux Shell

Prepared by:
Kudzaishe Majeza
Junior Network Engineer – VxLEARN Networks

Mentor:
Titus Majeza
Senior Network Engineer

Date: 05 March 2026

Introduction

This lab introduces the Linux command-line environment (the shell) and covers essential operations including navigation, directory management, file redirection, hidden files, copying, deleting, and moving files. The lab was performed on the CyberOps Workstation Virtual Machine logged in as the user 'analyst'.

Recommended Equipment

- CyberOps Workstation virtual machine (analyst / cyberops)

Part 1: Shell Basics

Step 1: Access the Command Line

- Logged on to the CyberOps Workstation VM as 'analyst' using the password 'cyberops'.
- Opened the Terminal Emulator by clicking the terminal icon in the Dock at the bottom of the VM screen.



Step 2: Display Manual Pages from the Command Line

- Displayed the man page for the man command:

```
[analyst@secOps ~]$ man man
```

A screenshot of a terminal window titled "Terminal - analyst@secOps:~". The window shows the output of the command "man man". The output is formatted as a manual page with sections: NAME, SYNOPSIS, and DESCRIPTION. The terminal window has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The background of the terminal is black with white text. The text is as follows:

```
MAN(1)                                Manual pager utils                                MAN(1)

NAME

    man - an interface to the system reference manuals

SYNOPSIS

    man [man options] [section] page ...] ...
    man -k [apropos options] regexp ...
    man -K [man options] [section] term ...
    man -f [whatis options] page ...
    man -l [man options] file ...
    man -w|-W [man options] page ...

DESCRIPTION

    man is the system's manual pager. Each page argument given to man is normally the name of a program, utility or function. The manual page associated with each of these arguments is then found and displayed. A section, if provided, will direct man to look only in that section of the manual. The default action is to search in all of the available sections following a pre-defined order (see DEFAULTS), and to show only the first page found, even if page exists in several sections.

    The table below shows the section numbers of the manual followed by the
```

Manual page man(1) line 1 (press h for help or q to quit)

Question: Name a few sections that are included in a man page.

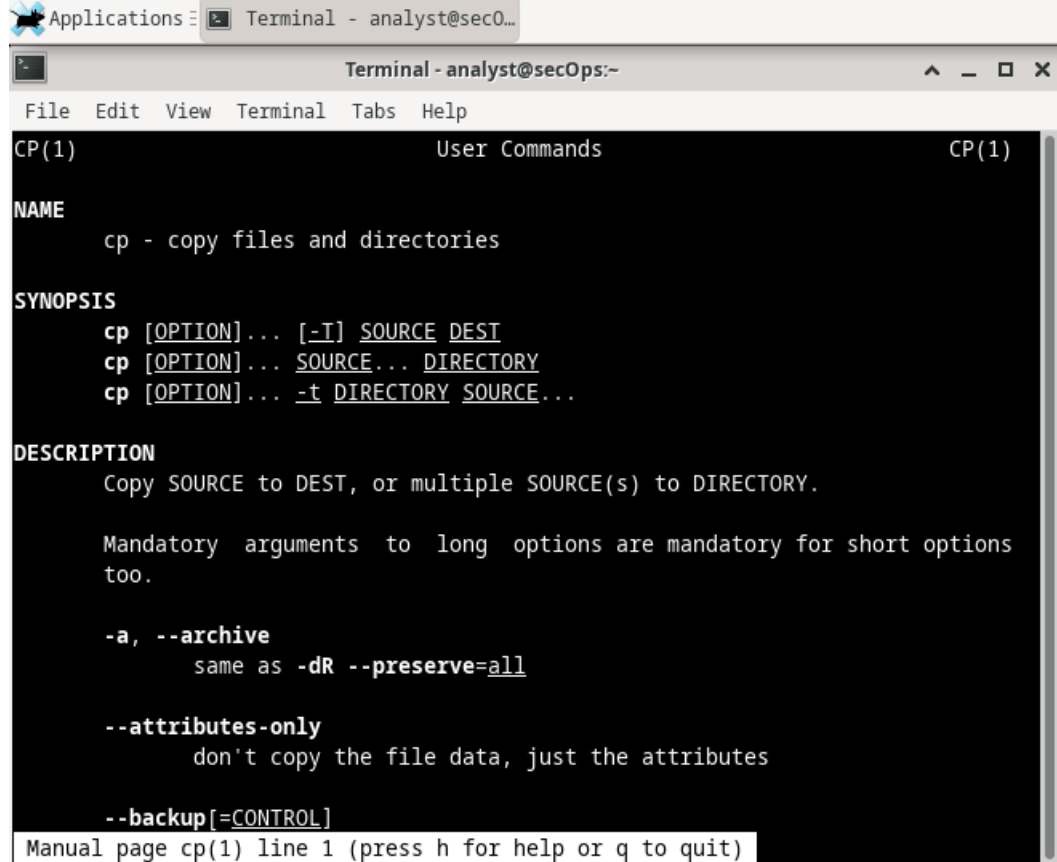
A man page typically includes the following sections:

- NAME – the name of the command and a brief description
- SYNOPSIS – the command syntax and available options
- DESCRIPTION – a detailed explanation of what the command does
- OPTIONS – a list of all flags/options the command accepts
- EXAMPLES – practical usage examples
- SEE ALSO – references to related commands or documentation
- BUGS – known issues
- AUTHOR – author information

b. Typed 'q' to exit the man page.

c. Displayed the man page for the cp command:

```
[analyst@secOps ~]$ man cp
```



```
CP(1)                                User Commands                                CP(1)

NAME
    cp - copy files and directories

SYNOPSIS
    cp [OPTION]... [-I] SOURCE DEST
    cp [OPTION]... SOURCE... DIRECTORY
    cp [OPTION]... -t DIRECTORY SOURCE...

DESCRIPTION
    Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.

    Mandatory arguments to long options are mandatory for short options
    too.

    -a, --archive
        same as -dR --preserve=all

    --attributes-only
        don't copy the file data, just the attributes

    --backup[=CONTROL]

Manual page cp(1) line 1 (press h for help or q to quit)
```

Question: What is the function of the cp command?

The cp (copy) command is used to copy files and directories from one location to another on the local filesystem. The original source file remains unchanged and an identical copy is created at the destination.

Basic syntax: cp [OPTION]... SOURCE DEST

Example: cp file.txt /home/analyst/backup/

Question: What command would you use to find out more about the pwd command? What is the function of the pwd command?

Command: man pwd

Function: pwd stands for 'Print Working Directory'. It displays the full absolute path of the current working directory in the filesystem. This is useful for confirming your current location when navigating the directory tree.

Example output: /home/analyst

Step 3: Create and Change Directories

a. Displayed the current directory using pwd:

```
[analyst@secOps ~]$ pwd
/home/analyst
```

Question: What is the current directory?

The current directory is /home/analyst (the home directory of the analyst user).

b. Navigated to /home/analyst:

```
[analyst@secOps ~]$ cd /home/analyst
```

c. Listed files and folders with ls -l:

```
[analyst@secOps ~]$ ls -l
total 20
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

d. Created three new directories using mkdir:

```
[analyst@secOps ~]$ mkdir cyops_folder1
[analyst@secOps ~]$ mkdir cyops_folder2
[analyst@secOps ~]$ mkdir cyops_folder3
```

e. Verified folder creation with ls -l:

```
[analyst@secOps ~]$ ls -l
total 32
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder1
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder3
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

f. Changed to cyops_folder3:

```
[analyst@secOps ~]$ cd /home/analyst/cyops_folder3
[analyst@secOps cyops_folder3]$
```

Question: Which folder are you in now?

The current directory is /home/analyst/cyops_folder3 as confirmed by the prompt [analyst@secOps cyops_folder3]\$.

Challenge: Type 'cd ~' and describe what happens. Why did this happen?

After typing cd ~, the shell returned to the home directory /home/analyst and the prompt changed back to [analyst@secOps ~]\$.

Why: The tilde (~) is a shell shortcut that always expands to the current user's home directory (defined by the \$HOME environment variable). Therefore, 'cd ~' is equivalent to 'cd /home/analyst'. This provides a quick way to return home from any location in the filesystem.

g. Created cyops_folder4 inside cyops_folder3 using an absolute path:

```
[analyst@secOps ~]$ mkdir
/home/analyst/cyops_folder3/cyops_folder4
```

h. Verified folder creation:

```
[analyst@secOps cyops_folder3]$ ls -l
total 4
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:53 cyops_folder4
```

i. Examined the . and .. directory entries using ls -la:

```
[analyst@secOps cyops_folder3]$ ls -la
total 12
drwxr-xr-x  3 analyst analyst 4096 Mar  6 07:53 .
drwx----- 17 analyst analyst 4096 Mar  6 07:31 ..
drwxr-xr-x  2 analyst analyst 4096 Mar  6 07:53 cyops_folder4
```

j. Changed directory to cyops_folder3:

```
[analyst@secOps ~]$ cd /home/analyst/cyops_folder3
```

k. Ran cd . (current directory):

```
[analyst@secOps cyops_folder3]$ cd .
[analyst@secOps cyops_folder3]$
```

Question (k): What happens?

Nothing visible happens. The prompt remains [analyst@secOps cyops_folder3]\$. The single dot '.' refers to the current directory itself, so 'cd .' simply changes into the same directory you are already in — effectively a no-op.

l. Ran cd .. (parent directory):

```
[analyst@secOps cyops_folder3]$ cd ..
[analyst@secOps ~]$
```

Question (l): What happens?

The shell moved up one level to the parent directory. Since cyops_folder3 is inside /home/analyst, running 'cd ..' returned us to /home/analyst, shown by the prompt reverting to [analyst@secOps ~]\$.

What would be the current directory if you issued cd .. at [analyst@secOps ~]\$?

The current directory would become /home (one level above /home/analyst). Prompt would show: [analyst@secOps home]\$

What would be the current directory if you issued cd .. at [analyst@secOps home]\$?

The current directory would become / (the root directory, one level above /home). Prompt would show: [analyst@secOps /]\$

What would be the current directory if you issued `cd ..` at `[analyst@secOps ~]$`?

The current directory would remain `/` (the root directory). The root `/` has no parent directory — it is the top of the filesystem hierarchy. `'cd ..'` from root simply stays at `/`.

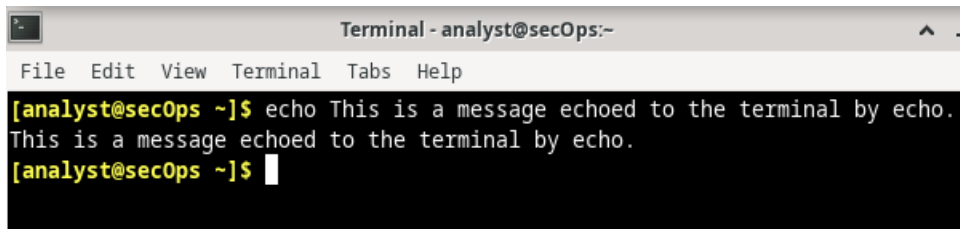
```
[analyst@secOps cyops_folder3]$ cd .  
[analyst@secOps cyops_folder3]$ cd ..
```

Step 4: Redirect Outputs

a. Navigated back to the home directory:

```
[analyst@secOps ~]$ cd /home/analyst/  
[analyst@secOps ~]$ pwd  
/home/analyst
```

b. Used `echo` to output a message to the terminal:

A terminal window titled "Terminal - analyst@secOps:~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The prompt is [analyst@secOps ~]\$. The command echo This is a message echoed to the terminal by echo. is entered, and the output This is a message echoed to the terminal by echo. is displayed. The prompt [analyst@secOps ~]\$ is shown again with a cursor.

```
Terminal - analyst@secOps:~  
File Edit View Terminal Tabs Help  
[analyst@secOps ~]$ echo This is a message echoed to the terminal by echo.  
This is a message echoed to the terminal by echo.  
[analyst@secOps ~]$
```

c. Redirected `echo` output to a text file using `>`:

```
[analyst@secOps ~]$ echo This is a message echoed to the terminal by echo. > some_text_file.txt
```

Question: No output was shown. Is that expected? Explain.

Yes, this is expected. The `>` (redirect) operator intercepts the output of the `echo` command and writes it directly to the specified file (`some_text_file.txt`) instead of sending it to the terminal (`stdout`). Because all output was redirected to the file, nothing is displayed on screen. This is the normal and intended behavior of output redirection in Linux.

d. & e. Verified file creation and viewed contents:

```
[analyst@secOps ~]$ ls -l
total 36
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder1
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder2
drwxr-xr-x 3 analyst analyst 4096 Mar  6 07:53 cyops_folder3
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst  50 Mar  6 09:09 some_text_file.txt
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
[analyst@secOps ~]$ cat some_text_file.txt
This is a message echoed to the terminal by echo.
```

f. & g. Redirected a different message to the same file and verified:

```
[analyst@secOps ~]$ echo This is a DIFFERENT message, once again echoed to the t
erminal by echo. > some_text_file.txt
[analyst@secOps ~]$ cat some_text_file.txt
This is a DIFFERENT message, once again echoed to the terminal by echo.
```

Question: What happened to the text file? Explain.

The original content of some_text_file.txt was completely overwritten (erased) and replaced with the new message.

Explanation: The single > (redirect) operator always overwrites the destination file. If the file already exists, its entire contents are first truncated to zero before the new data is written. This is destructive — the old content is lost permanently unless backed up. If the file does not exist, it is created.

Step 5: Redirect and Append to a Text File

a. Appended a new line to the file using >>:

```
[analyst@secOps ~]$ echo This is another line of
text. It will be APPENDED to the output file. >>
some_text_file.txt
```

b. Verified the file contents:

```
[analyst@secOps ~]$ cat some_text_file.txt
This is a DIFFERENT message, once again echoed to the
terminal by echo.
This is another line of text. It will be APPENDED to
the output file.
```

Question: What happened to the text file? Explain.

A second line was added to the end of the file without removing the existing content. The original message is preserved and the new message appears below it.

Explanation: The double >> (append redirect) operator opens the destination file in append mode — it places the cursor at the end of the file and writes the new data there. Unlike >, the >> operator does NOT erase existing content. This is very useful for writing log files or accumulating data over multiple operations.

```
[analyst@secOps ~]$ echo This is another line of text. It will be APPENDED to the output file. >> some_text_file.txt
[analyst@secOps ~]$ cat some_text_file.txt
This is a DIFFERENT message, once again echoed to the terminal by echo.
This is another line of text. It will be APPENDED to the output file.
```

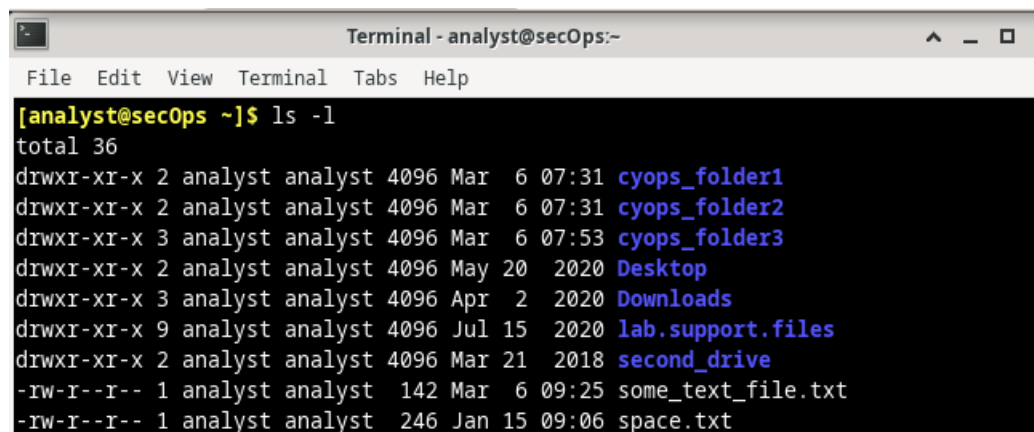
Step 6: Work with Hidden Files in Linux

b. Listed files with `ls -l` (hidden files NOT shown):

```
[analyst@secOps ~]$ ls -l
```

Question: How many files are displayed?

Approximately 9 items are displayed (the 3 cyops_folders, Desktop, Downloads, lab.support.files, second_drive, some_text_file.txt, and space.txt). Hidden files beginning with a dot '.' are not shown by `ls -l`.



```
Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ ls -l
total 36
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder1
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder2
drwxr-xr-x 3 analyst analyst 4096 Mar  6 07:53 cyops_folder3
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 142 Mar  6 09:25 some_text_file.txt
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

c. Listed all files including hidden with ls -la:

```
[analyst@secOps ~]$ ls -la
```

```
[analyst@secOps ~]$ ls -la
total 140
drwx----- 17 analyst analyst 4096 Mar  6 09:09 .
drwxr-xr-x  3 root    root    4096 Mar 20  2018 ..
-rw-----  1 analyst analyst 1086 Jan 15 17:07 .bash_history
-rw-r--r--  1 analyst analyst  21 Feb  7  2018 .bash_logout
-rw-r--r--  1 analyst analyst  57 Feb  7  2018 .bash_profile
-rw-r--r--  1 analyst analyst  97 Jan 15 17:54 .bashrc
-rw-r--r--  1 analyst analyst 141 Feb  7  2018 .bashrc_stock
drwxr-xr-x  8 analyst analyst 4096 Mar 25  2020 .cache
drwxr-xr-x 10 analyst analyst 4096 Jan 11 18:30 .config
drwxr-xr-x  2 analyst analyst 4096 Mar  6 07:31 cyops_folder1
drwxr-xr-x  2 analyst analyst 4096 Mar  6 07:31 cyops_folder2
drwxr-xr-x  3 analyst analyst 4096 Mar  6 07:53 cyops_folder3
drwxr-xr-x  2 analyst analyst 4096 May 20  2020 Desktop
-rw-r--r--  1 analyst analyst  23 Mar 23  2018 .dmrc
drwxr-xr-x  3 analyst analyst 4096 Apr  2  2020 Downloads
drwx----- 3 analyst analyst 4096 Mar 22  2018 .gnupg
-rw-----  1 analyst analyst 2520 Mar 24  2020 .ICEauthority
drwxr-xr-x  2 analyst analyst 4096 Mar 24  2018 .idlerc
drwxr-xr-x  9 analyst analyst 4096 Jul 15  2020 lab.support.files
-rw-----  1 analyst analyst  89 May 20  2020 .lessht
drwxr-xr-x  3 analyst analyst 4096 Mar 22  2018 .local
drwx----- 5 analyst analyst 4096 Mar 24  2018 .mozilla
```

Question: How many MORE files are displayed than before? Explain.

Several additional files and directories are revealed — typically 8 or more extra hidden entries. The -a flag tells ls to include all entries whose names begin with a dot '.' (hidden files and directories). These include configuration files such as .bash_profile, .bashrc, .bash_logout, .bash_history, .viminfo, the special '.' (current directory) entry, and the '..' (parent directory) entry.

Question: Is it possible to hide entire directories by adding a dot before the name? Are there any directories in the output of ls -la?

Yes — adding a dot prefix to any directory name will hide it from normal ls output just like regular files. Directories are fully visible only when using ls -la or ls -a.

In the ls -la output you can see hidden directories such as:

- . (current directory itself)
- .. (parent directory)
- .config (hidden configuration directory)
- .dbus, .cache, .local (common hidden system directories)

These are denoted by 'd' as the first character in the permission string.

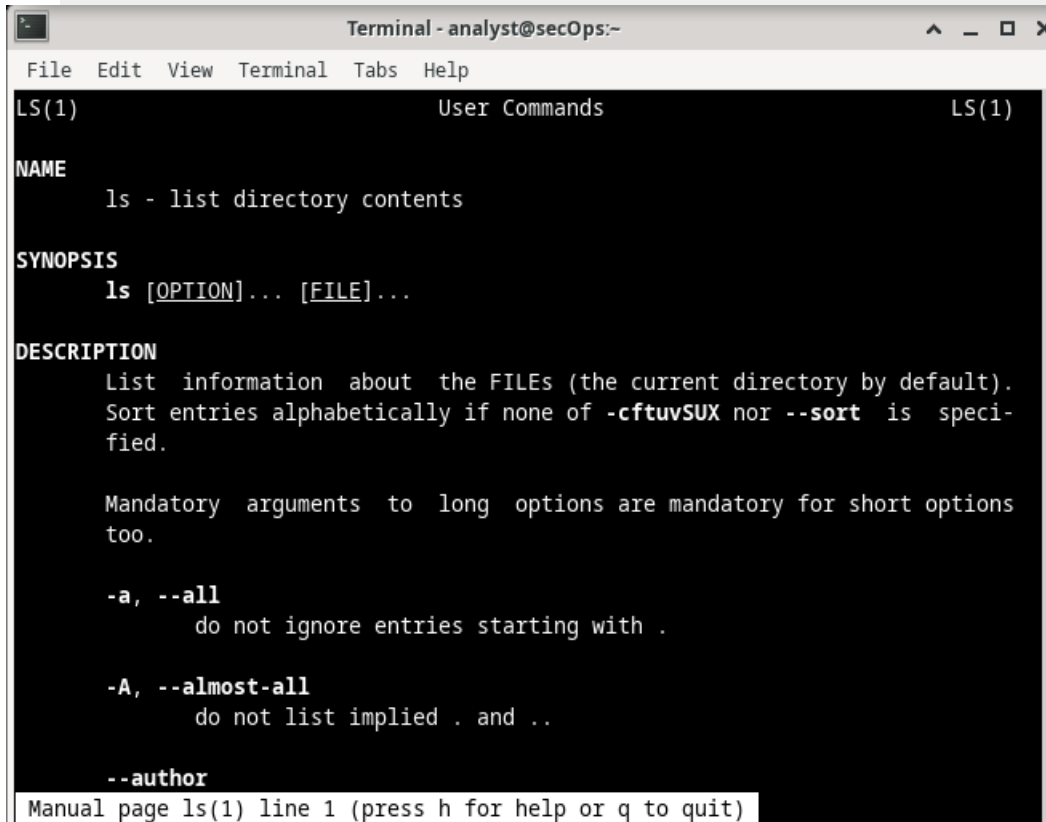
Question: Give three examples of hidden files shown in the output of `ls -la`.

Three examples of hidden files typically seen in the analyst home directory:

1. `.bash_history` – stores the history of commands typed in the bash shell
2. `.bashrc` – bash shell configuration script run for interactive shells
3. `.bash_profile` – user-specific environment setup script run at login

d. Viewed the man page for `ls`:

```
[analyst@secOps ~]$ man ls
```



The screenshot shows a terminal window titled "Terminal - analyst@secOps:~". The terminal displays the man page for the `ls` command. The page is divided into sections: **NAME**, **SYNOPSIS**, and **DESCRIPTION**. The **NAME** section states "ls - list directory contents". The **SYNOPSIS** section shows the command format: `ls [OPTION]... [FILE]...`. The **DESCRIPTION** section explains that `ls` lists information about files in the current directory by default, sorting entries alphabetically unless specified otherwise. It also lists several options: `-a, --all` (do not ignore entries starting with `.`), `-A, --almost-all` (do not list implied `.` and `..`), and `--author`. At the bottom of the terminal, a status bar reads "Manual page ls(1) line 1 (press h for help or q to quit)".

```
LS(1) User Commands LS(1)

NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the FILES (the current directory by default).
  Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
  fied.

  Mandatory arguments to long options are mandatory for short options
  too.

  -a, --all
      do not ignore entries starting with .

  -A, --almost-all
      do not list implied . and ..

  --author

Manual page ls(1) line 1 (press h for help or q to quit)
```

Part 2: Copying, Deleting, and Moving Files

Step 1: Copying Files

a. Copied some_text_file.txt to cyops_folder2:

```
[analyst@secOps ~]$ cp some_text_file.txt  
cyops_folder2/
```

Question: What are the source and destination files? (use full paths)

Source : /home/analyst/some_text_file.txt

Destination : /home/analyst/cyops_folder2/some_text_file.txt

The cp command creates an exact duplicate of the source at the destination. The original file at /home/analyst/some_text_file.txt remains untouched.

b. Verified the copy in cyops_folder2:

```
[analyst@secOps ~]$ ls cyops_folder2/  
some_text_file.txt
```

c. Verified the original still exists in the home directory:

```
[analyst@secOps ~]$ ls -l  
total 36  
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder1  
drwxr-xr-x 2 analyst analyst 4096 Mar  6 09:36 cyops_folder2  
drwxr-xr-x 3 analyst analyst 4096 Mar  6 07:53 cyops_folder3  
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop  
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads  
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files  
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive  
-rw-r--r-- 1 analyst analyst 142 Mar  6 09:25 some_text_file.txt  
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

Step 2: Deleting Files and Directories

a. Deleted some_text_file.txt from the home directory with rm:

```
[analyst@secOps ~]$ rm some_text_file.txt  
[analyst@secOps ~]$ ls -l  
total 32  
drwxr-xr-x 2 analyst analyst 4096 Mar  6 07:31 cyops_folder1  
drwxr-xr-x 2 analyst analyst 4096 Mar  6 09:36 cyops_folder2  
drwxr-xr-x 3 analyst analyst 4096 Mar  6 07:53 cyops_folder3  
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop  
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads  
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files  
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive  
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

b. Deleted the cyops_folder1 directory and all its contents with rm -r:

```
[analyst@secOps ~]$ rm -r cyops_folder1
[analyst@secOps ~]$ ls -l
total 28
drwxr-xr-x 2 analyst analyst 4096 Mar  6 09:36 cyops_folder2
drwxr-xr-x 3 analyst analyst 4096 Mar  6 07:53 cyops_folder3
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

Step 3: Moving Files and Directories

a. Moved some_text_file.txt from cyops_folder2 back to the home directory:

```
[analyst@secOps ~]$ mv cyops_folder2/some_text_file.txt .
[analyst@secOps ~]$ ls -l cyops_folder2/
total 0
[analyst@secOps ~]$ ls -l /home/analyst/
total 32
drwxr-xr-x 2 analyst analyst 4096 Mar  6 09:49 cyops_folder2
drwxr-xr-x 3 analyst analyst 4096 Mar  6 07:53 cyops_folder3
drwxr-xr-x 2 analyst analyst 4096 May 20 2020 Desktop
drwxr-xr-x 3 analyst analyst 4096 Apr  2 2020 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 15 2020 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 142 Mar  6 09:36 some_text_file.txt
-rw-r--r-- 1 analyst analyst 246 Jan 15 09:06 space.txt
```

Question: What command did you use to accomplish the task?

Command used: mv cyops_folder2/some_text_file.txt .

- mv = move (rename) a file or directory
- cyops_folder2/some_text_file.txt = relative source path
- . = destination is the current working directory (/home/analyst)

The mv command moves the file from its source location to the destination. Unlike cp, the original is removed after the move is complete — the file now only exists in /home/analyst.

Reflection

Question: What are the advantages of using the Linux command line?

The Linux command line (shell) offers numerous advantages over a graphical interface:

1. Speed and Efficiency – Experienced users can execute complex tasks far faster by typing commands than by navigating menus and clicking through windows.
2. Automation and Scripting – Commands can be combined into shell scripts to automate repetitive tasks, batch-process files, and schedule jobs (cron).
3. Remote Access – The shell works over SSH on remote systems where no GUI is available, making it essential for server and network administration.
4. Precise Control – Every aspect of the operating system is accessible via commands, providing finer control than most GUIs allow.
5. Low Resource Usage – The CLI consumes minimal CPU and memory compared to graphical desktop environments, freeing resources for other tasks.
6. Powerful Redirection and Piping – The ability to redirect output (>, >>) and chain commands with pipes (|) enables powerful data-processing workflows.
7. Consistency – Shell commands behave consistently across different Linux distributions and versions, making skills highly transferable.
8. Logging and Auditing – Commands typed in the shell are saved in `.bash_history`, providing a record of actions useful for troubleshooting and security auditing.

Command Reference Summary

The table below summarises all key commands covered in this lab.

Command	Syntax / Example	Description
man	man <command>	Display the manual page for a command
pwd	pwd	Print the current working directory
cd	cd /path cd ~ cd ..	Change directory (absolute, home, parent)
ls	ls -l ls -la	List directory contents (-l long, -a all)

mkdir	mkdir folder_name	Create a new directory
echo	echo text	Print text to stdout or a file
>	echo text > file.txt	Redirect output, overwriting the file
>>	echo text >> file.txt	Redirect output, appending to the file
cat	cat file.txt	Display file contents in the terminal
cp	cp source destination	Copy files or directories
rm	rm file rm -r dir	Remove files (-r for recursive/directories)
mv	mv source destination	Move or rename files and directories
.	cd .	Refers to the current directory
..	cd ..	Refers to the parent directory
~	cd ~	Shortcut to the current user's home dir